

Система за препоръчване на GitHub репозиторията

Изготвил: Екатерина Хамбарлийска
Факултетен номер: 8MI3400767

7 юни 2026 г.

Съдържание

1 Резюме	3
2 Проблем и хипотеза	3
2.1 Проблем	3
2.2 Хипотеза	4
3 Какво прави системата	4
4 Данни	4
5 Архитектура на системата	5
5.1 Зареждане и обработка на данните	5
5.2 Генериране на семантични представяния	5
5.3 Изграждане на графова структура	5
5.4 Генериране на графови представяния	7
5.5 Изчисляване на показатели за популярност	8
5.6 Хибриден модул за препоръки	8
5.7 Потребителски интерфейс	8
5.8 Сравнение с голям езиков модел	8
6 Реализация	8
7 Оценяване на системата	9
7.1 Back-Translation Evaluation	9
7.2 Approximate Ground Truth Evaluation	10

7.3 Precision@5	10
7.4 Recall@5	10
7.5 HitRate@5	11
7.6 Mean Reciprocal Rank (MRR)	11
7.7 Coverage	11
7.8 Diversity	12
7.9 Human Evaluation	12
7.10 NDCG@5	13
8 Резултати	13
8.1 Автоматична оценка	13
8.2 Coverage и Diversity	14
8.3 Human Evaluation	15
8.4 Обобщение	16
9 Заключение	16
10 Използвана информация	17

1 Резюме

Проектът разработва хибридна content-based препоръчваща система за GitHub репозиторията, която генерира препоръки въз основа на вече избрани от потребителя хранилища.

Системата комбинира три подхода: семантично сходство между описанията чрез векторни представяния, графови представяния на репозиторията чрез Node2Vec и хибриден модел, който включва и показатели за популярност.

Има потребителски интерфейс, разработен със Streamlit, както и възможност за сравнение на резултатите с локален голям езиков модел чрез Ollama и Mistral.

Основната цел на проекта е да демонстрира как текстовите и структурните представяния на репозиторията могат да бъдат използвани за генериране на препоръки в среди, в които липсват потребителски оценки, кликове или други форми на взаимодействие.

2 Проблем и хипотеза

2.1 Проблем

В GitHub съществуват множество публични репозиторията, което затруднява намирането на най-релевантното съдържание само чрез ръчно търсене. Допълнително предизвикателство е, че различните проекти могат да бъдат описани по различен начин, въпреки че принадлежат към една и съща тематична област.

Класическите препоръчващи системи често използват потребителски оценки, история на взаимодействията или матрици потребител-обект. В използвания набор от данни подобна информация не е налична. Разполагаме основно с характеристики на самите репозиторията. Това налага използването на content-based подход, а не методи, базирани на потребителски взаимодействия.

2.2 Хипотеза

Описанията на репозиторитата съдържат достатъчно семантична информация за определяне на сходство между проектите. Освен това връзките между сходни репозиторита могат да бъдат представени чрез графова структура. Предполага се, че комбинирането на семантично сходство, графови представяния и показатели за популярност ще доведе до по-качествени и по-балансирані препоръки в сравнение с използването на всеки от подходите отделно или с използването на голям езиков модел.

3 Какво прави системата

Вход: списък от GitHub репозиторита, избрани от потребител.

Изход: списък с най-подходящи и сходни хранилища, подредени според степента на сходство. По-популярните проекти получават по-висок приоритет.

4 Данни

Използван е Kaggle набор от данни за GitHub репозиторита. След почистване проектът работи с 500 хранилища. Ограничение е, че липсват потребителски рейтинги, кликове и реални етикети за релевантност.

За всяко GitHub хранилище са налични:

- **Name** – име на репозитория;
- **Description** – кратко текстово описание на проекта;
- **CreatedAt** – дата на създаване;
- **Stars** – брой потребители, отбелязали проекта като интересен;
- **Forks** – брой разклонения на проекта;
- **Issues** – брой отворени задачи и проблеми.

Name	Description	CreatedAt	Stars	Forks	Issues
tensorflow	Open Source Machine Learning Framework for Everyone	2015-11-07	177	889	206

Таблица 1: Примерен запис от използвания набор от GitHub репозиторита.

5 Архитектура на системата

Архитектурата включва следните основни компоненти:

5.1 Зареждане и обработка на данните

От набора от данни се извличат основните характеристики на GitHub репозиторитата. Данните се почистват и подготвят за последваща обработка.

5.2 Генериране на семантични представяния

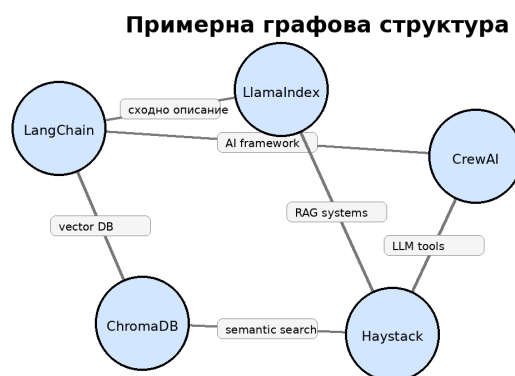
Описанията на репозиторитата се преобразуват във векторни представяния чрез предварително обучен езиков модел SentenceTransformer. Получените вектори позволяват измерване на семантично сходство между различни проекти.

5.3 Изграждане на графова структура

Графът се използва за представяне на връзките между сходни GitHub хранилища. За да бъдат приложени графови ембединги, репозиторитата се представят като граф на сходство. Всеки възел в графа представлява едно GitHub репозитори, а ребрата описват семантична близост между репозиторитата. Графът се изгражда като за всяко репозитори се избират петте най-близки репозиторита според косинусовата близост. Между текущия възел и всеки от тези пет най-близки съседни се създава ребро. По този начин не се използ-

ва фиксиран праг за сходство, а всяко репозитори се свързва с най-сходните си съседи. Полученият граф е неориентиран и претеглен:

- **Възли (Nodes):** GitHub хранилища.
- **Ребра (Edges):** връзки между хранилища, създадени при високо семантично сходство между техните описания.
- **Тежест на реброто (Edge Weight):** стойност на cosine similarity между embedding представянията на описанията.
- **Цел:** да се уловят структурни зависимости между сходни проекти, които не винаги могат да бъдат открити само чрез директно текстово сравнение.



Възлите представят GitHub хранилища, а ребрата - сходство между тях.

Фигура 1: Пример за графова структура между сходни хранилища.

Тъй като графът е неориентиран, финалната степен на даден възел не е задължително точно 5. Всеки възел избира до пет най-близки съседи, но може също да бъде избран като съсед от други възли. Затова в крайния граф степените на възлите варират. В реализирания вариант графът съдържа 500 възела, 1878 ребра, средна степен 7.512, минимална степен 5 и максимална степен 29.

Както се вижда на Фигура 2, повечето възли имат сравнително ниска степен, докато малка част от репозиторитата функционират като по-свързани възли в графа. Това се дължи на факта, че някои проекти са избирани като близки съседи от множество други репозиторита. Цветът на възлите визуализира тяхната степен, а пространственото групиране показва наличието на тематични клъстери от сходни проекти.

5.4 Генериране на графови представления

7

представяне, което описва неговата позиция и връзки в мрежата.

5.5 Изчисляване на показатели за популярност

За всяко репозитори се изчислява оценка за популярност на базата на броя звезди, разклонения и отворени задачи.

5.6 Хибриден модул за препоръки

Системата комбинира:

- семантично сходство;
- графово сходство;
- показатели за популярност.

Получава се крайна оценка, използвана за подреждане на препоръките.

5.7 Потребителски интерфейс

Потребителят избира между едно и пет репозиторията. Системата изгражда общ профил на интересите и генерира списък с най-подходящите препоръки.

5.8 Сравнение с голям езиков модел

Като допълнителна оценка се използва локален езиков модел чрез Ollama (Mistral), който генерира препоръки върху същите входни данни. Резултатите се сравняват с тези на разработената система.

6 Реализация

Системата е реализирана като Python проект със Streamlit потребителски интерфейс. Потребителят избира от 1 до 5 репозиторията от случайно генериран списък, след което системата генерира Топ 5 препоръки за всеки от трите основни

метода: Semantic, Graph и Hybrid. Интерфейсът показва препоръките и визуализира наличните метрики за оценка.

Метод	Техническа реализация	Идея
Semantic	SentenceTransformer all-MiniLM-L6-v2 и cosine similarity	Репозиторита с подобни описания получават по-висок ранг.
Graph	Граф на репозиторита и Node2Vec embeddings	Репозиторита, близки в графовото пространство, се приемат за свързани.
Hybrid	$0.45 \cdot \text{semantic_similarity} + 0.35 \cdot \text{graph_similarity} + 0.20 \cdot \text{popularity_score}$	Комбиниращ текстово сходство, графов сигнал и популярност.
Ollama	Локален Mistral модел чрез Ollama	Използва се само като LLM baseline за качествено сравнение.

$$\text{popularity_score} = 0.50 \cdot \text{normalized_Stars} + 0.30 \cdot \text{normalized_Forks} + 0.20 \cdot \text{normalized_Issues}$$

Таблица 2: Основни методи в системата.

7 Оценяване на системата

Използваните данни не съдържат потребителски оценки, история на взаимодействия, кликове или реални етикети за релевантност. Затова не е възможно директно използване на класически методи за оценяване, базирани на реален ground truth. Поради това са приложени няколко допълващи се стратегии за оценяване, които измерват различни аспекти на качеството на препоръките.

7.1 Back-Translation Evaluation

За оценяване на способността на системата да разпознава семантично сходни описания е използван подходът Back-Translation Evaluation. Избрани са 20 репозиторита от набора от данни. Описанието на всеки проект първо се превежда от английски

на японски език, а след това обратно на английски. Получава се перифразирана версия на текста със запазен смисъл, но различна формулировка.

Новото описание се използва като заявка към препоръчващата система. Тъй като е известно от кои репозиторията произлиза описанието, оригиналният проект се приема за очакван резултат. На тази база се изчисляват HitRate@5 и Mean Reciprocal Rank (MRR), които показват дали и на каква позиция системата успява да открие най-близкото репозитори.

7.2 Approximate Ground Truth Evaluation

За всяка заявка се определя приблизително релевантно множество чрез избор на Top 10 съседни на съответното множество. Това множество се използва като approximate ground truth при автоматичното оценяване. След това всеки метод генерира Top 5 препоръки и се измерва каква част от тях попадат в предварително определеното релевантно множество. Необходимо е да се отбележи, че този подход не представлява напълно независима оценка, тъй като релевантността се определя чрез семантични ембединги. Поради това резултатите следва да се разглеждат като сравнителен ориентир между различните методи.

7.3 Precision@5

Precision@5 измерва каква част от първите пет препоръки са релевантни.

$$Precision@5 = \frac{\text{брой релевантни препоръки в Top 5}}{5}$$

По-високата стойност показва по-точни препоръки и по-малко нерелевантни резултати.

7.4 Recall@5

Recall@5 измерва каква част от всички релевантни репозитория са открити в Top 5 препоръките.

$$Recall@5 = \frac{\text{брой релевантни препоръки в Top 5}}{\text{всички релевантни препоръки}}$$

Тъй като релевантното множество съдържа 10 елемента, максималната стойност на Recall@5 е ограничена до 0.5.

Precision@5 и Recall@5 се изчисляват спрямо приблизително релевантно множество, определено чрез семантични съседи. Поради липсата на реални етикети за релевантност тези метрики се използват като сравнителен ориентир между методите, а не като абсолютно измерване на качеството.

7.5 HitRate@5

HitRate@5 измерва дали в Top 5 препоръките присъства поне един релевантен резултат.

$$HitRate@5 = \begin{cases} 1, & \text{ако има поне един релевантен резултат в Top 5} \\ 0, & \text{иначе} \end{cases}$$

След това стойността се усреднява за всички тестови заявки.

7.6 Mean Reciprocal Rank (MRR)

MRR измерва колко рано в списъка се появява първият релевантен резултат.

$$Reciprocal\ Rank = \frac{1}{\text{позиция на първия релевантен резултат}}$$

Ако първият релевантен резултат е на първо място, стойността е 1.0. Ако е на второ място – 0.5. Ако няма релевантен резултат, стойността е 0.

7.7 Coverage

Coverage измерва каква част от всички налични репозитори-та се появяват поне веднъж в препоръките.

$$Coverage = \frac{\text{брой уникални препоръчани репозиторията}}{\text{общ брой репозиторията}}$$

Високата стойност показва, че системата използва по-голяма част от каталога и не препоръчва постоянно едни и същи популярни проекти.

7.8 Diversity

Diversity измерва разнообразието на препоръките в рамките на един списък.

За всяка препоръка се използват *embedding* представянията на препоръчаните репозиторията и се изчислява средното *cosine similarity* между всички двойки елементи. След това *diversity* се определя като:

$$Diversity = 1 - \text{average pairwise similarity}$$

По-висока стойност означава по-разнообразни препоръки.

7.9 Human Evaluation

Освен автоматичната оценка е проведена и ръчна оценка. За избрани входни репозиторията са генерирани Top 5 препоръки от всеки метод. Всяка препоръка е оценена по следната скала:

- 0 – нерелевантна;
- 1 – слабо релевантна;
- 2 – релевантна;
- 3 – силно релевантна.

На база на тези оценки са изчислени Average Human Score и NDCG@5.

7.10 NDCG@5

NDCG@5 (Normalized Discounted Cumulative Gain) измерва не само дали препоръките са релевантни, но и дали най-релевантните резултати са поставени в началото на списъка.

Метриката отчита както качеството на препоръките, така и тяхното подреждане, което я прави особено подходяща за оценяване на препоръчващи системи.

8 Резултати

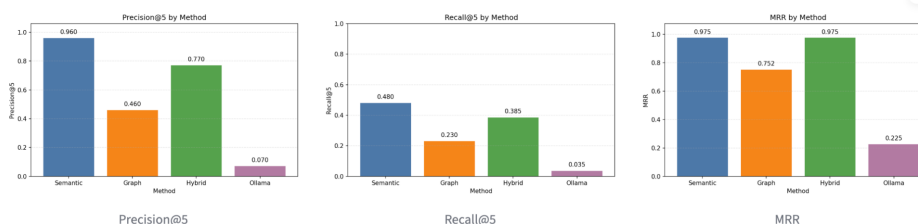
8.1 Автоматична оценка

Фигура 3 представя резултатите от автоматичната оценка чрез Precision@5, Recall@5, HitRate@5 и MRR.

- Semantic постига най-високи стойности по всички автоматични метрики.
- Hybrid се представя близко до Semantic и значително по-добре от Graph.
- Graph има по-ниски резултати, но запазва добра способност за откриване на релевантни репозиторията.
- Ollama baseline показва най-ниски стойности, тъй като не използва предварително изчислени embeddings и графова структура.

Retrieval Accuracy

Method	Precision@5	Recall@5	HitRate@5	MRR
Semantic	0.96	0.48	1	0.975
Graph	0.46	0.23	0.9	0.7517
Hybrid	0.77	0.385	1	0.975
Ollama	0.07	0.035	0.25	0.225



Фигура 3: Автоматична оценка на методите.

8.2 Coverage и Diversity

Coverage измерва каква част от каталога се използва от даден метод, а Diversity измерва разнообразието на препоръките.

- Graph има най-висока Coverage (0.098), което показва по-добро покритие на каталога.
- Semantic и Hybrid постигат сходни стойности.
- Ollama използва много малка част от наличните репозиторията.
- Ollama има най-висока Diversity, но това е за сметка на по-ниска релевантност.
- Graph постига по-висока Diversity от Semantic и Hybrid.

Coverage and Diversity

Coverage shows how much of the repository catalog each method explores. Diversity shows whether the Top 5 recommendations are varied or too similar to each other.

Method	Coverage	Unique Recommended	Total Repositories	Average Diversity
Semantic	0.088	44	500	0.5207
Graph	0.098	49	500	0.5748
Hybrid	0.094	47	500	0.5322
Ollama	0.012	6	500	0.7735

Фигура 4: Coverage и Diversity на методите.

8.3 Human Evaluation

За избрани заявки е извършена ръчна оценка на препоръките по скала от 0 до 3. Използвани са две метрики: Average Human Score и NDCG@5.

Резултатите показват, че:

- Graph получава най-висока средна човешка оценка (2.18).
- Hybrid постига най-висок NDCG@5 (0.9429), което показва най-добро подреждане на релевантните резултати.
- Semantic също се представя силно.
- Ollama остава значително под останалите методи.

Human Relevance Evaluation

Human relevance uses manual ratings from 0 to 3. NDCG@5 rewards methods that place highly relevant repositories near the top.

Method	Average Human Score	NDCG@5
Semantic	2.14	0.9067
Graph	2.18	0.8778
Hybrid	2.16	0.9429
Ollama	0.7	0.8774

Фигура 5: Резултати от Human Evaluation.

8.4 Обобщение

Автоматичната оценка показва предимство на Semantic подхода, което е очаквано поради използването на семантични съседи като приблизително релевантно множество. Graph подходът осигурява по-голямо разнообразие и по-добро покритие на каталога. Human Evaluation показва, че Hybrid моделът предлага най-добър баланс между релевантност, поддръждане и разнообразие на препоръките, което го прави най-подходящия основен метод в разработената система.

9 Заключение

Графовият подход чрез Node2Vec работи добре като допълнителен източник на информация. Той не винаги превъзхожда семантичния модел при автоматични метрики, базирани на текстово сходство, но добавя различна перспектива чрез структурните връзки в графа. Това се вижда най-вече при Coverage и Diversity, където графовият модел показва по-добро покритие на каталога и по-разнообразни препоръки.

Като най-подходяща оценка за реализирания алгоритъм се използва комбинация от метрики: HitRate@5 и MRR за проверка дали системата открива очакваните резултати, NDCG@5 и Human Score за оценка на качеството и поддръждането на препоръките, както и Coverage и Diversity за анализ на поведението на графовия модел. Резултатите показват, че хибридният модел предлага най-добър практически баланс между релевантност, структурна близост и популярност.

Следователно графовите ембединги чрез Node2Vec са полезни за задачата, особено когато се комбинират със семантично сходство и популярност. Ограничение остава липсата на реална ground truth информация. Поради това автоматичните резултати трябва да се разглеждат като индикативни, а не като окончателно доказателство за качество. За по-надеждна оценка са необходими повече човешки оценки, реални потребителски сценарии или данни за взаимодействия.

10 Използвана информация

1. Kaggle dataset: <https://www.kaggle.com/datasets/donbarbos/github-repos>
2. SentenceTransformers documentation: <https://www.sbert.net/>
3. Grover, A., Leskovec, J. (2016). *node2vec: Scalable Feature Learning for Networks*.
4. Streamlit documentation: <https://streamlit.io/>
5. Ollama documentation: <https://ollama.com/>